



Software Engineering

UNDERSTANDING TEST-DRIVEN DEVELOPMENT
(TDD)

dr. inz Abdulla Juwaied

Learning Objectives

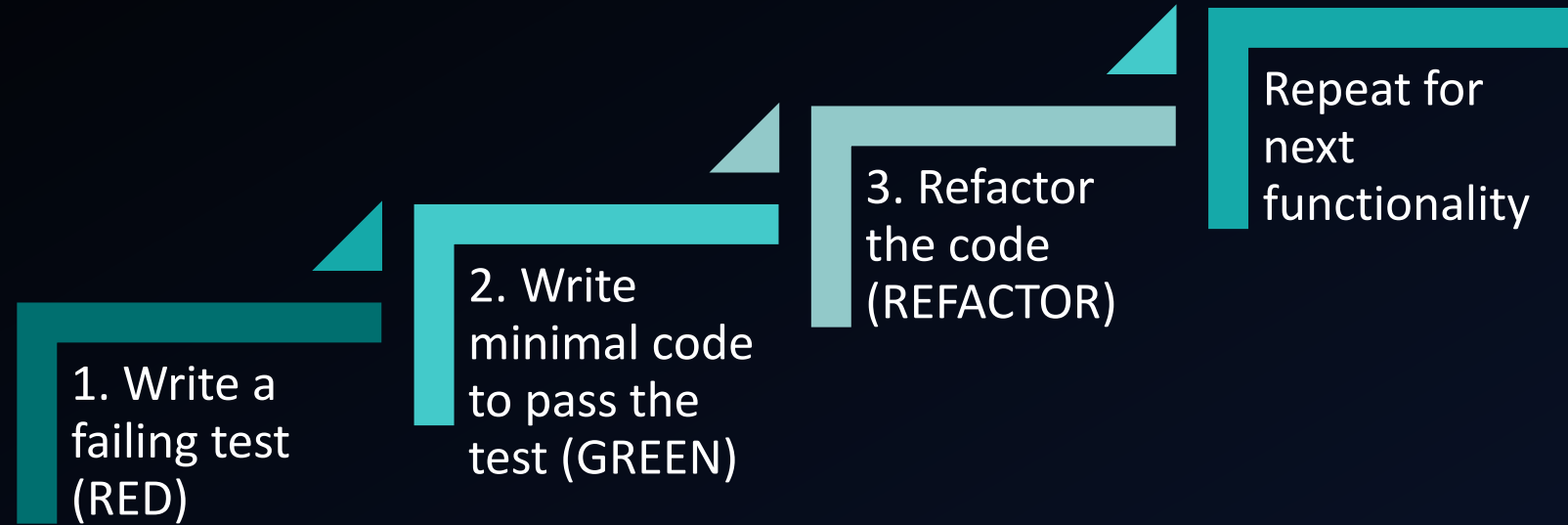
By completing this kata, you should understand:

- The TDD cycle (Red-Green-Refactor)
- The three laws of TDD
- How design emerges from tests
- The importance of refactoring
- How to write clean, readable tests
- How to let tests drive your design decisions
- The value of small, incremental steps

Understanding Test-Driven Development (TDD)

- Test-Driven Development (TDD) is an approach to program development where you interleave testing and code development. Tests are written before code, and passing the tests is the critical driver of development.
- Key Principles:
 - You develop code incrementally, along with a test for that increment
 - You don't move on to the next increment until the code passes its test
 - TDD was introduced as part of agile methods (like Extreme Programming), but can be used in any development process

The TDD Process Cycle



Visual representation:

- RED: Write a test that fails
- GREEN: Make the test pass with minimal code
- REFACTOR: Clean up the code while keeping tests green

Understanding Test-Driven Development (TDD)

- Test-Driven Development (TDD) is an approach to program development where you interleave testing and code development. Tests are written before code, and passing the tests is the critical driver of development.
- Key Principles:
 - You develop code incrementally, along with a test for that increment
 - You don't move on to the next increment until the code passes its test
 - TDD was introduced as part of agile methods (like Extreme Programming), but can be used in any development process

The Bowling Game Kata : Lab Task

Apply TDD methodology to build a bowling score calculator step-by-step, focusing on:

- Following the three laws of TDD
- Incremental development
- Continuous refactoring
- Clean, readable code

Functional Requirements

You must create a class named **Game** with two methods:

- `roll(pins: int)`
 1. Called each time the player rolls a ball
 2. The parameter `pins` represents the number of pins knocked down
- `score() : int`
 1. Called only at the very end of the game
 2. Returns the total score for that game

Bowling Scoring Rules

Game Structure:

- The game consists of **10 frames**
- In each frame, the player has **two opportunities** to knock down 10 pins
- The score for the frame is the total number of pins knocked down, **plus bonuses** for strikes and spares

Spare:

- When the player knocks down all 10 pins in **two tries**
- Bonus:** The number of pins knocked down by the **next roll**
- Example: Frame 3 has 6 + 4 (spare), next roll is 5 → Frame 3 score = 10 + 5 = 15

Strike:

- When the player knocks down all 10 pins on the **first try**
- Bonus:** The value of the **next two balls** rolled
- Example: Frame 1 has strike (10), next two rolls are 3 and 4 → Frame 1 score = 10 + 3 + 4 = 17

Tenth Frame (Special Case):

- A player who rolls a spare or strike is allowed to roll **extra balls** to complete the frame
- However, **no more than three balls** can be rolled in the tenth frame

Initial Design Session

Before jumping into code, let's think about the design:

Initial thoughts (from the presentation):

- We clearly need the `Game` class ✓
- A game has 10 frames → Maybe a `Frame` class?
- A frame has 1 or 2 rolls → Maybe a `Roll` class?
- The tenth frame is different (2 or 3 rolls) → Maybe a `TenthFrame` class?
- The score function must iterate through frames
- The score for a spare or strike depends on the frame's successor

BUT WAIT!

In TDD, we **don't design everything up front**. We let the design **emerge** through the tests and refactoring. So let's start simple and see where the tests take us.

Step-by-Step TDD Implementation

Step 1: Project Setup

Create the test class first:

```
import junit.framework.TestCase;

public class BowlingGameTest extends TestCase {
}
```

Run the program and verify you get the error:

No tests found in BowlingGameTest

Good! This is our first "failure" - we need a test.

Step 2: First Test - Gutter Game

RED Phase: Write a failing test

Goal: Test the case where no pins are knocked down in all 20 rolls (the worst possible game).

```
public void testGutterGame() throws Exception {  
    Game g = new Game();  
    for (int i = 0; i < 20; i++)  
        g.roll(0);  
    assertEquals(0, g.score());  
}
```

Run the test → It fails because _____?

GREEN Phase: Write minimal code to pass

```
public class Game {  
    public void roll(int pins) {  
    }  
  
    public int score() {  
        return 0;  
    }  
}
```

REFACTOR Phase: Nothing to refactor yet.

Step 3: Second Test - All Ones

RED Phase: Write the next test

Goal: Test the case where one pin is knocked down on each roll.

```
public void testAllOnes() throws Exception {  
    Game g = new Game();  
    for (int i = 0; i < 20; i++)  
        g.roll(1);  
    assertEquals(20, g.score());  
}
```

Run the test → Fails with expected:<20> but was:<0>

GREEN Phase: Make it pass

We need to actually track the rolls and calculate the score:

```
public class Game {  
    private int score = 0;  
  
    public void roll(int pins) {  
        score += pins;  
    }  
  
    public int score() {  
        return score;  
    }  
}
```

Run the tests → Both pass!

Step 3: Second Test - All Ones

REFACTOR Phase: Clean up the tests

Notice the duplication in our tests:

- Game creation is duplicated
- The roll loop is duplicated

Run the tests → Still pass!

```
public class BowlingGameTest extends TestCase {
    private Game g;

    protected void setUp() throws Exception {
        g = new Game();
    }

    private void rollMany(int n, int pins) {
        for (int i = 0; i < n; i++)
            g.roll(pins);
    }

    public void testGutterGame() throws Exception {
        rollMany(20, 0);
        assertEquals(0, g.score());
    }

    public void testAllOnes() throws Exception {
        rollMany(20, 1);
        assertEquals(20, g.score());
    }
}
```

Step 4: Third Test - One Spare

RED Phase: Write the test

Goal: Test a spare in the first frame.

Run the test → Fails with
expected:<16> but
was:<13>

GREEN Phase: First attempt

We're tempted to use a flag to remember the previous roll, but this feels wrong. The current design has a problem:

- `roll()` calculates score, but the name doesn't imply that
- `score()` doesn't calculate score, but the name implies it does

Comment out the test temporarily and rethink.

```
public void testOneSpare() throws Exception {  
    g.roll(5);  
    g.roll(5); // spare  
    g.roll(3);  
    rollMany(17, 0);  
    assertEquals(16, g.score()); // 10 + 3 (bonus) + 3 = 16  
}
```

```
public class Game {  
    private int[] rolls = new int[21];  
    private int currentRoll = 0;  
  
    public void roll(int pins) {  
        rolls[currentRoll++] = pins;  
    }  
  
    public int score() {  
        int score = 0;  
        for (int i = 0; i < rolls.length; i++)  
            score += rolls[i];  
        return score;  
    }  
}
```

Step 4: Third Test - One Spare

Run existing tests → Still pass!

Now **uncomment the spare test** → Still fails with expected:<16> but was:<13>

The problem: We're just summing all rolls, but we need to iterate **frame by frame**:

```
public int score() {  
    int score = 0;  
    int frameIndex = 0;  
    for (int frame = 0; frame < 10; frame++) {  
        score += rolls[frameIndex] + rolls[frameIndex + 1];  
        frameIndex += 2;  
    }  
    return score;  
}
```

Run tests → Still fails! We need to detect spares



```
public int score() {  
    int score = 0;  
    int frameIndex = 0;  
    for (int frame = 0; frame < 10; frame++) {  
        if (rolls[frameIndex] + rolls[frameIndex + 1] == 10) { // spare  
            score += 10 + rolls[frameIndex + 2];  
            frameIndex += 2;  
        } else {  
            score += rolls[frameIndex] + rolls[frameIndex + 1];  
            frameIndex += 2;  
        }  
    }  
    return score;  
}
```

Run tests → All pass!

Step 4: Third Test - One Spare

REFACTOR Phase: Clean up

The code has issues:

- Ugly comment in the conditional
- `frameIndex` is not a great name (but it's okay)
- Duplicated expressions

```
public int score() {
    int score = 0;
    int frameIndex = 0;
    for (int frame = 0; frame < 10; frame++) {
        if (isSpare(frameIndex)) {
            score += 10 + spareBonus(frameIndex);
            frameIndex += 2;
        } else {
            score += sumOfBallsInFrame(frameIndex);
            frameIndex += 2;
        }
    }
    return score;
}

private boolean isSpare(int frameIndex) {
    return rolls[frameIndex] + rolls[frameIndex + 1] == 10;
}

private int spareBonus(int frameIndex) {
    return rolls[frameIndex + 2];
}

private int sumOfBallsInFrame(int frameIndex) {
    return rolls[frameIndex] + rolls[frameIndex + 1];
}
```

```
public void testOneSpare() throws Exception {
    rollSpare();
    g.roll(3);
    rollMany(17, 0);
    assertEquals(16, g.score());
}

private void rollSpare() {
    g.roll(5);
    g.roll(5);
}
```

Run tests → All pass!

Also refactor the test to remove the ugly comment.

Step 5: Fourth Test - One Strike

RED Phase:

```
public void testOneStrike() throws Exception {
    g.roll(10); // strike
    g.roll(3);
    g.roll(4);
    rollMany(16, 0);
    assertEquals(24, g.score()); // 10 + 3 + 4 (bonus) + 3 + 4 = 24
}
```

Run test → Fails with expected:<24> but was:<17>

GREEN Phase:

```
public int score() {
    int score = 0;
    int frameIndex = 0;
    for (int frame = 0; frame < 10; frame++) {
        if (rolls[frameIndex] == 10) { // strike
            score += 10 + rolls[frameIndex + 1] + rolls[frameIndex + 2];
            frameIndex++;
        } else if (isSpare(frameIndex)) {
            score += 10 + spareBonus(frameIndex);
            frameIndex += 2;
        } else {
            score += sumOfBallsInFrame(frameIndex);
            frameIndex += 2;
        }
    }
    return score;
}
```

Run tests → All pass!

Step 5: Fourth Test - One Strike

REFACTOR Phase:

```
public int score() {
    int score = 0;
    int frameIndex = 0;
    for (int frame = 0; frame < 10; frame++) {
        if (isStrike(frameIndex)) {
            score += 10 + strikeBonus(frameIndex);
            frameIndex++;
        } else if (isSpare(frameIndex)) {
            score += 10 + spareBonus(frameIndex);
            frameIndex += 2;
        } else {
            score += sumOfBallsInFrame(frameIndex);
            frameIndex += 2;
        }
    }
    return score;
}

private boolean isStrike(int frameIndex) {
    return rolls[frameIndex] == 10;
}

private int strikeBonus(int frameIndex) {
    return rolls[frameIndex + 1] + rolls[frameIndex + 2];
}
```

Also refactor the test:

```
public void testOneStrike() throws Exception {
    rollStrike();
    g.roll(3);
    g.roll(4);
    rollMany(16, 0);
    assertEquals(24, g.score());
}

private void rollStrike() {
    g.roll(10);
}
```

Run tests → All pass!

Run tests → All pass!

Step 6: Fifth Test - Perfect Game

RED Phase:

Goal: Test the ultimate game - all strikes (score = 300).

```
public void testPerfectGame() throws Exception {  
    rollMany(12, 10);  
    assertEquals(300, g.score());  
}
```

Note: 12 rolls because the 10th frame gets 2 bonus balls after a strike.

Run test → It passes immediately!

Your code already handles this case correctly!

List Of tasks [OPTIONAL]

- **Smart Thermostat Controller:** Write a logic engine that receives temperature readings and decides whether to turn on "Heating," "Cooling," or "Idle" based on a target threshold and a "hysteresis" (to prevent rapid switching).
- **Traffic Light Controller (State Machine):** Model a 4-way intersection. Implement the logic for transitions (Green → Yellow → Red) ensuring that two perpendicular directions are never Green at the same time.
- **K-Nearest Neighbours (KNN) Distance Engine or K-MEANS:** Implement a function that calculates the distance between a new data point and a training set using **Euclidean** and **Manhattan** metrics.
- **Simple Linear Regression Trainer:** Given a set of (x, y) coordinates, calculate the slope (m) and intercept (b) using the Least Squares method.

For the final Task: Use the "Ping-Pong" Method

To make these effective for students, suggest the **Ping-Pong Pair Programming** approach:

- 1.Student A:** Writes a failing test for a small requirement.
- 2.Student B:** Writes the code to pass that test, then writes the *next* failing test.
- 3.Student A:** Writes the code to pass that test, then refactors the code, then writes the *next* failing test.
- 4.Repeat.**

This keeps both students engaged and ensures that the "Navigator" isn't just watching, but actively thinking about the next requirement.